

BVMT TFT — Pre-Training Checklist

Final validation before launching model training

Esprit School of Business — Tunisia | March 2026

Current Status — Data Quality: GO

Data quality check: ALL PASS

116,626 price rows | 71 stocks | 2016-2025

98% join coverage between `daily_prices` and `computed_indicators`

All key stocks have 500+ trading days

Zero impossible price values remaining

Decision: keep stocks with 500+ days even if early years are missing

You have 7 steps to complete before launching training. Steps 1-4 are mandatory. Steps 5-7 are recommended for thesis quality. Each step has exact code to run in `feature_engineering.ipynb` or a new validation notebook.

STEP 1 — Feature sanity checks — verify the built dataset

WHY: Before training, confirm the CSV exported by `feature_engineering.ipynb` is exactly what you expect. Wrong shapes, unexpected NULLs, or incorrect column types will silently corrupt training without any error message.

Add this validation cell at the end of `feature_engineering.ipynb`

```
import pandas as pd
from pathlib import Path

df = pd.read_csv('data/features/tft_features.csv')
df['seance'] = pd.to_datetime(df['seance'])

print('=== FEATURE SANITY REPORT ===')
print(f'Shape           : {df.shape}')
print(f'Stocks           : {df["ticker_id"].nunique()}')
print(f'Date range       : {df["seance"].min().date()} to {df["seance"].max().date()}')
print(f'NULLs total       : {df.isnull().sum().sum()}')
print()

# Check 1: No NULLs in any feature column
null_cols = df.isnull().sum()
null_cols = null_cols[null_cols > 0]
if len(null_cols) == 0:
    print('PASS: No NULLs in any column')
else:
    print('FAIL: NULLs found:')
    print(null_cols)

# Check 2: time_idx must be unique per stock
dups = df.groupby(['ticker_id', 'time_idx']).size()
dups = dups[dups > 1]
if len(dups) == 0:
    print('PASS: time_idx is unique per stock')
else:
    print(f'FAIL: {len(dups)} duplicate time_idx values')

# Check 3: RSI must be in [0, 100]
bad_rsi = df[(df['rsi_14'] < 0) | (df['rsi_14'] > 100)]
print(f'PASS: RSI valid' if len(bad_rsi)==0 else f'FAIL: {len(bad_rsi)} bad RSI')

# Check 4: MA20 must be positive
bad_ma = df[df['ma_20'] <= 0]
print(f'PASS: MA20 positive' if len(bad_ma)==0 else f'FAIL: {len(bad_ma)} bad MA20')

# Check 5: close_price must be positive
bad_close = df[df['close_price'] <= 0]
print(f'PASS: close_price positive' if len(bad_close)==0 else f'FAIL: {len(bad_close)} bad close')

# Check 6: target balance
bal = df['direction_7d'].value_counts(normalize=True)
```

```

up_pct = bal.get(1, 0) * 100
print(f'Class balance : UP={up_pct:.1f}% DOWN={100-up_pct:.1f}%')
if up_pct > 65:
    print('WARNING: Class imbalance > 65/35 - use class_weight in training')
elif up_pct < 35:
    print('WARNING: Majority DOWN - check target computation')
else:
    print('PASS: Class balance acceptable')

# Check 7: min rows per stock
min_rows = df.groupby('ticker_id').size().min()
print(f'Min rows/stock : {min_rows}')
print('PASS: all stocks have enough rows' if min_rows >= 500 else 'WARNING: some stocks < 500 rows')

```

PASS conditions:

NULLs = 0 | time_idx unique | RSI in [0,100] | MA20 > 0 | close > 0
 Class balance: UP between 40% and 65%
 Min rows per stock: >= 500

STEP 2 — Leakage check — walk-forward split validation

WHY: Data leakage is the single most dangerous mistake in financial ML. It happens when future information accidentally enters the training set. A leaky model shows 80%+ accuracy in training but fails completely on live data.

In your setup, leakage can happen in two ways:

- Target leakage: direction_7d computed using shift(-7) — if not done per-group, future prices from a different stock contaminate the target
- Feature leakage: indicators computed using data from after the prediction date

Add this leakage validation cell in feature_engineering.ipynb

```

# LEAKAGE CHECK 1: Confirm shift(-7) was applied per ticker group
# Pick one stock and manually verify the last 10 rows have NaN target
sample = df[df['ticker'] == 'AMEN BANK'].sort_values('seance').tail(15)
print('Last 15 rows of AMEN BANK - last 7 should be dropped (NaN target):')
print(sample[['seance', 'close_price', 'direction_7d']].to_string())
# If you see direction_7d values in the last 7 rows → leakage!

# LEAKAGE CHECK 2: Confirm split boundaries are clean
train = df[df['seance'] < '2024-01-01']
val = df[(df['seance'] >= '2024-01-01') & (df['seance'] < '2025-01-01')]
test = df[df['seance'] >= '2025-01-01']

```

```

print(f'Train: {len(train):,} rows {train["seance"].min().date()} to
{train["seance"].max().date()}')
print(f'Val : {len(val):,} rows {val["seance"].min().date()} to
{val["seance"].max().date()}')
print(f'Test : {len(test):,} rows {test["seance"].min().date()} to
{test["seance"].max().date()}')

# LEAKAGE CHECK 3: No overlap between splits
train_dates = set(train['seance'])
val_dates = set(val['seance'])
test_dates = set(test['seance'])
overlap_tv = train_dates & val_dates
overlap_vt = val_dates & test_dates
print(f'Train/Val overlap : {len(overlap_tv)} dates (must be 0)')
print(f'Val/Test overlap : {len(overlap_vt)} dates (must be 0)')

# LEAKAGE CHECK 4: Target not available for future dates
# The last 7 trading days of each stock should NOT be in the dataset
last_dates = df.groupby('ticker')['seance'].max()
max_global = df['seance'].max()
gap_days = (max_global - last_dates).dt.days
print(f'Max date in dataset: {max_global.date()}')
print(f'Stocks with data up to last date: {(gap_days == 0).sum()}')
# If any stock has rows within 7 days of max_date → check if target is NaN

```

CRITICAL: If train/val or val/test overlap is > 0 → stop and fix before training.
The 7-day gap between last price row and prediction target is handled by dropna()
in feature_engineering Cell 5 — confirm dropna was run after computing direction_7d.

STEP 3 — Class imbalance check and decision

WHY: BVMT is a developing market with strong bull periods. If 65%+ of all days show UP direction, the model will learn to always predict UP and achieve 65% accuracy without learning anything useful. You need to know the imbalance before choosing a training strategy.

Run this in a new cell after loading tft_features.csv

```

# Overall balance
overall = df['direction_7d'].value_counts(normalize=True)
print(f'Overall: UP={overall.get(1,0)*100:.1f}% DOWN={overall.get(0,0)*100:.1f}%')

# Balance per year — important: imbalance may vary
df['year'] = df['seance'].dt.year
yearly = df.groupby('year')['direction_7d'].mean() * 100
print('\nUP% by year (>60 means bull year):')
for yr, pct in yearly.items():
    flag = ' ← BULL' if pct > 60 else (' ← BEAR' if pct < 40 else '')
    print(f' {yr}: {pct:.1f}%{flag}')

```

```
# Balance per stock — find most imbalanced
stock_bal = df.groupby('ticker')['direction_7d'].mean() * 100
print('\nMost imbalanced stocks (UP%):')
print(stock_bal.sort_values(ascending=False).head(10).to_string())
print(stock_bal.sort_values(ascending=True).head(10).to_string())
```

UP% result	What it means	What to do in train_tft.py
45-55%	Well balanced	No action needed — train normally
55-65%	Mild imbalance	Add class_weight='balanced' to loss function
>65%	Strong imbalance	Use weighted CrossEntropy: weight=[UP_ratio, DOWN_ratio]
<35%	Something wrong	Check shift(-7) direction — may be inverted

STEP 4 — Stock filtering — apply your 500-day rule

WHY: Your decision is correct — keep stocks with 500+ days. But you need to apply it consistently in `feature_engineering.ipynb` so that stocks with insufficient history do not corrupt the sliding window. A stock with only 200 days cannot form a proper 60-day encoder window for most of its history.

Confirm this filter exists in `feature_engineering.ipynb` Cell 7

```
# This must be in Cell 7 of feature_engineering.ipynb
stock_counts = df.groupby('ticker')['seance'].count()
valid_tickers = stock_counts[stock_counts >= 500].index
df = df[df['ticker'].isin(valid_tickers)]
print(f'Stocks after 500-day filter: {df["ticker"].nunique()}')
print(f'Removed: {stock_counts[stock_counts < 500].index.tolist()}')
```

Additionally apply this refinement for **CARTHAGE CEMENT** — your data shows it only starts in 2021. It has 1,256 days which passes the 500-day threshold, but it has no data for 2016-2020. This is fine for training — TFT will simply learn from 2021-2025 for this stock. No action needed.

Stocks to explicitly exclude regardless of day count

```
# Add this exclusion list to Cell 7
EXCLUDE_TICKERS = {
    'HANNIBAL LEASE', # 54% zero-volume — too illiquid
    'SIMPAN',         # 52% zero-volume — too illiquid
    'CEREALIS',       # only 1 year of data
```

```

'AMS',          # only 65 trading days
'SERVICOM',     # only 123 days, ends 2021
'BNA ASSURANCES', # only 248 days, starts 2025
'OFFICEPLAST',  # only 248 days, starts 2025
}
df = df[~df['ticker'].isin(EXCLUDE_TICKERS)]
print(f'Stocks after exclusion: {df["ticker"].nunique()}')

```

STEP 5 — Minimal ablation plan before full training

WHY: Running one full training job takes 1-2 hours on Lightning AI. If it fails or gives bad results, you have lost the session. A minimal ablation runs 3 quick experiments first (each 5-10 minutes) to confirm the setup works before committing to full training.

Ablation 1 — Smoke test (run this first, 5 minutes)

Train on only 3 stocks (AMEN BANK, SFBT, ARTES) for 3 epochs. Goal: confirm no errors, loss decreases, model saves correctly.

```

# In train_tft.py, add this at the top for smoke test
SMOKE_TEST = True # set to False for full training

if SMOKE_TEST:
    SMOKE_STOCKS = [0, 1, 2] # first 3 ticker_ids
    df = df[df['ticker_id'].isin(SMOKE_STOCKS)]
    MAX_EPOCHS = 3
    print('SMOKE TEST MODE: 3 stocks, 3 epochs')

```

Ablation 2 — Single year validation (10 minutes)

Train on 2016-2022, validate on 2023 only. If val_loss decreases → model is learning. If val_loss stays at 0.693 (random baseline) → feature engineering problem.

Ablation 3 — Full training with early stopping (1-2 hours)

Only run this after Ablation 1 and 2 both pass. Use 2016-2023 train, 2024 validation, 2025 test. This is the real training run.

Expected val_loss progression (healthy training):
 Epoch 1: ~0.690 (just below random baseline of 0.693)
 Epoch 5: ~0.675
 Epoch 10: ~0.660
 Early stop: ~0.645 (best model saved here)

If val_loss never goes below 0.690 after 5 epochs → learning rate too low or features wrong

If val_loss drops to 0.500 in epoch 1 → likely data leakage, check immediately

STEP 6 — Go/No-Go criteria for launching training

Run through this checklist in order. All items must be YES before you start the Lightning AI session.

#	Criterion	How to verify
1	tft_features.csv exists and has > 80,000 rows	Cell: print(len(df))
2	Zero NULLs in all feature columns	Step 1 sanity check PASS
3	time_idx is unique per stock (no duplicate entries)	Step 1 sanity check PASS
4	Train/Val/Test splits have zero date overlap	Step 2 leakage check: both overlaps = 0
5	Class balance is between 35% and 65% UP	Step 3 balance check
6	Smoke test (3 stocks, 3 epochs) runs without error	Step 5 Ablation 1
7	val_loss decreases in smoke test (not stuck at 0.693)	Watch training output
8	Model checkpoint file saved to /models/ folder	Check file exists after smoke test

STOP conditions — do not proceed to full training if any of these occur:

val_loss = 0.693 after 5+ epochs → features are not informative, check engineering

val_loss < 0.500 in epoch 1 → data leakage, check split boundaries

CUDA out of memory → reduce BATCH_SIZE from 128 to 64 or 32

KeyError or shape mismatch → check TimeSeriesDataSet column names match CSV

STEP 7 — Record results for thesis

After training completes, record these metrics in your research_notes.md file. These numbers go directly into Chapter 5 of your thesis.

Metric	Where to find it	Thesis location
Best val_loss	Training output log	Chapter 5, Table 5.1
Directional accuracy on 2025 test	evaluate_tft.ipynb	Chapter 5, Table 5.1

Training epochs before early stop	Training output log	Chapter 4, Section 4.2
Class balance (UP%)	Step 3 output	Chapter 5, Section 5.1
Number of stocks in final training set	Step 4 output	Chapter 4, Section 4.1
Total training rows	Step 1 shape output	Chapter 4, Section 4.1